

Datalab 实验报告

杨鼎铉 2023200224

实验结果：得分：37

```
• ydx@admin:~/~datalab2026$ make && ./btest
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
1      1      0      bitAnd
1      1      0      bitXor
2      2      0      byteSwap
3      3      0      logicalShift
4      4      0      float_i2f
4      4      0      leftBitCount
4      4      0      floatPower2
4      4      0      floatScale2
3      3      0      reverse
2      2      0      samesign
4      4      0      logtwo

• ydx@admin:~/~datalab2026$ python3 test.py
rm -f *.o btest fshow ishow *~ yacctab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd      1/1:      PASS
bitXor      1/1:      PASS
samesign    2/2:      PASS
logtwo      4/4:      PASS
byteSwap    4/4:      PASS
reverse     3/3:      PASS
logicalShift 3/3:      PASS
leftBitCount 4/4:      PASS
float_i2f    4/4:      PASS
floatScale2 4/4:      PASS
float64_f2i 3/3:      PASS
floatPower2 4/4:      PASS
Total points: 37
```

1.bitAnd

- 本题要求仅用取反和按位或实现按位与。按位运算逐位独立，故只需考察单个二进制位：两位同时为 1 时结果才为 1，这等价于"两位中不存在 0"。"某位为 0"可用取反表示，"至少有一位为 0"用或运算表示，整体再取反即得"不存在 0"。这正是德摩根定律 $x \& y = \sim(\sim x | \sim y)$ 的直接应用。共使用 4 个运算符，满足不超过 7 个的限制。

2.bitXor

- 异或的语义是"两位不同则为 1"，可等价拆分为两个条件的合取：至少有一位为 1（即 $x | y$ ），且不同时为 1（即 $\sim(x \& y)$ ），故 $x \wedge y = (x | y) \& \sim(x \& y)$ 。由于本题禁用按位或，再次利用德摩根定律将其改

写为 $x|y=\sim(\sim x\&\sim y)$ ，代入后得到仅含取反与按位与的表达式 $\sim(\sim x\&\sim y)\&\sim(x\&y)$ ，共 7 个运算符，恰好达到上限。另一种常见写法 $\sim(\sim(x\&\sim y)\&\sim(\sim x\&y))$ 虽然结果正确，但需 8 个运算符，超出限制，故不采用。

3.samesign

- 题目将 0 单独归为一类，因此符号共有负、零、正三种取值，不能仅比较符号位。原因在于补码表示中 0 与正数的符号位同为 0，若直接比较符号位会误判 `samesign(0,1)` 为同号。
- 据此将问题分解为两种情形：其一，若两数中存在 0，则只有两数同时为 0 才算同号，用 `!x`、`!y` 判断是否为 0 并直接返回结果；其二，两数均非 0 时，仅剩正负两类，此时比较符号位即可。具体做法是先做异或，若两数符号位相同则异或结果的最高位为 0，否则为 1；再算术右移 31 位，使最高位扩展至全字，得到 0（同号）或 -1（异号）；最后用逻辑非将其转换为 1 或 0。先异或再移位只需一次移位，比分别移位后比较更省运算。共约 6 个运算符，满足不超过 12 个的限制。

4.logtwo

- 对正整数 v ， $\log_2 v$ 向下取整等于其最高有效位的位序号，因此本题实质是定位最高位的 1。逐位扫描需 32 次判断，效率低且运算符数量不够，故采用 **二分查找**：依次判断高 16 位、高 8 位、高 4 位、高 2 位、最低 1 位中是否含 1，若含 1 则说明最高位不低于该分界，将答案累加相应的位数，并把 v 右移相同位数，使高位部分下移，在缩小后的范围内继续查找；若不含 1 则答案不变，直接在低位部分继续。五次判断即可覆盖 32 位，因为 $16+8+4+2+1=31$ 恰好可表示所有位序号。
- 实现上用 $(v>>k)>0$ 判断高位是否含 1，结果为 1 或 0，再左移相应位数即得到本步应累加的值，从而避免使用条件语句。由于每步累加的 16、8、4、2、1 在二进制中各占一位且互不重叠，可用按位或代替加法（本题未提供加法运算符）。此处刻意使用 $v>>16>0$ 而非 $v>0xFFFF$ ，是为了避免使用超过 255 的大常数。共 21 个运算符，满足不超过 25 个的限制。

5.byteSwap（用异或构造掩码）

- 一个 32 位整数由 4 个字节组成，第 k 字节占据第 $8k$ 至 $8k+7$ 位，故字节号左移 3 位（等价于乘 8）即得到该字节的起始位号。
- 交换两个字节的直接做法是取出两字节、清空原位置、交叉写回，需要三步。本实现利用异或的性质进行优化：由 $a^{(a^b)}=b$ 可知，若令 $d=a^b$ ，则 **a 异或 d 得到 b ， b 异或 d 得到 a ，即同一个值可同时完成双向替换**。因此只需构造一个掩码，使两个目标字节位置上均为 d 、其余位置为 0，再与原数异或一次即可完成交换：目标字节按上述性质互换，其余字节因异或 0 而保持不变。
- 求 d 时，先将两次右移的结果异或，再与 `0xFF` 相与，只保留最低字节，较之分别取出两字节后再异或可少用一次与运算。当 n 等于 m 时

d 为 0，掩码为 0，原数不变，结果自然正确，无需特判。共 10 个运算符，满足不超过 17 个的限制。

补充异或性质：

基本性质

性质	公式	说明
和 0 异或不变	$a^0=a$	0 是"单位元"
自己和自己得 0	$a^a=0$	每一位都相同
和全 1 异或=取反	$a^{\sim}0=\sim a$	每一位都翻转
交换律	$a^b=b^a$	顺序无关
结合律	$(a^b)^c=a^(b^c)$	可以任意加括号
自反性（最重要）	$a^b^b=a$	异或两次回到原值

由基本性质推出的实用结论

1. 异或可以"撤销"

$$a^b^b=a^0=a$$

同一个数异或两次，等于什么都没做。加密里的一次一密就是这个原理：用密钥异或一次是加密，再异或一次就解密了。

2. 异或可以"交换"

若 $d=a^b$ ，则：

- $a^d=b$
- $b^d=a$

同一个 d，能把 a 变成 b，同时把 b 变成 a。这正是 byteSwap 用一次异或就完成两个字节交换的原理。

3. 异或是"按位可控翻转"

掩码位	效果
1	对应位翻转
0	对应位不变

想翻哪几位，就在掩码里把那几位置 1。logicalShift 用它把多补的 1 翻成 0；byteSwap 用它只改两个字节、其余不动。

4. 异或判断"是否相等"

$$a^b=0 \text{ 当且仅当 } a=b.$$

所以在不允许用 == 的题里，可以用 $!(a^b)$ 判断相等。sameSign 里就是用异或比较两个符号位是否相同。

5. 异或是"不进位的加法"

$$\begin{array}{r} 1101 \\ ^0111 \\ \hline 1010 \end{array}$$

每一位单独相加，不往高位进位
在硬件里，加法器正是用异或算本位结果、用与算进位。

6. 一串数异或，成对的会互相抵消

由交换律和 $a^a=0$ ：

$$3^5 \wedge 7 \wedge 3^5 = 7$$

经典应用：一组数里除了一个数只出现一次，其余都成对出现，把它们全部异或起来，剩下的就是那个单独的数。

6.reverse

- 本题允许使用循环，故采用逐位搬运的方式：每次取出 v 的最低位，追加到结果 r 的最低位，同时将 r 左移一位为新来的位腾出空间， v 右移一位丢弃已处理的位。最先取出的第 0 位在后续 31 次左移中被逐步推至最高位，最后取出的第 31 位停留在最低位，从而实现整体反转。
- 取最低位用 $v \& 1$ ，与 $0xFF$ 取字节同理，只是掩码宽度为 1 位。循环必须固定执行 32 次，而不能以 v 是否为 0 作为终止条件：若 v 的高位为 0，提前退出会使已搬运的位未被推至正确位置，例如 $\text{reverse}(1)$ 应得 $0x80000000$ ，提前退出则只得 1。由于合法运算符中不含 $<$ 与 $++$ ，采用计数器从 32 递减、以 $\text{while}(i)$ 判断是否归零的写法，仅需减法即可控制循环次数。函数参数为 `unsigned`，右移为逻辑右移，高位补 0，不受符号扩展干扰。循环体内共 5 个运算符，满足不超过 30 个的限制。

7.logicalShift

- C 语言中 **有符号数 (int) 的右移为算术右移**，高位补符号位（正数补 0，负数补 1），而本题要求逻辑右移（**unsigned 默认逻辑右移**），即高位一律补 0。两者的差异仅在于移位后高 n 位的取值：当 x 为负数时，算术右移会在高 n 位补入 n 个 1，这正是需要消除的部分；当 x 为非负数时，高位本就补 0，无需修正。
- 本实现的思路是精确构造出“多补的那几位”并用异或将其清除。先用 $x \& (1 \ll 31)$ 提取符号位，其余位清零；再对其算术右移 n 位，得到 $n+1$ 个 1（原符号位本身与补入的 n 个），左移一位去掉属于原数据的那一位，剩下的恰好是算术右移多补的 n 个 1；最后与 $x \gg n$ 异或，利用 $1 \wedge 1 = 0$ 将这些位翻转为 0，其余位异或 0 保持不变。
- 该写法对正数同样成立：符号位为 0 时修正值恒为 0，异或后结果不变，因此无需分支判断。 $n=0$ 时，符号位左移后被移出，修正值为 0，结果即 x 本身，也正确。此处未采用 $\sim 0 \ll (32-n)$ 之类的掩码构造方式，是因为 $n=0$ 时会产生移位 32 位的未定义行为。共 6 个运算符，满足不超过 20 个的限制。

8.leftBitCount

- 本题要求统计从最高位开始连续 1 的个数。实现时先对 x 取反，将“统计前导 1 的个数”转化为“统计前导 0 的个数”，这样每步的判断可简化为 $!(x \gg k)$ ，即判断高位若干位是否全为 0，无需每步额外取反。
- 统计前导 0 同样采用二分查找：依次判断高 16、8、4、2、1 位是否全为 0，若是则说明前导 0 的个数不少于该值，累加到结果中，并将 x 左移相同位数，把已统计过的 0 移出，在剩余部分继续判断；若否则不累加也不移位，在当前范围内继续细分。五步判断最多可统计到 $16+8+4+2+1=31$ 个前导 0。

- 由于五步之和上限为 31，当原数为全 1（即 x 取反后为 0）时结果会少 1，故最后补加 $!x$ ：此时 x 恒为 0， $!x$ 为 1，结果修正为 32；其余情况下 x 中必然含有 1，且左移只移出前导 0 而不会将该 1 移出，故 $!x$ 为 0，不影响结果。共 26 个运算符，满足不超过 50 个的限制。

9.float_i2f

一、问题分析

- 本题要求不借助浮点运算，手工完成 int 到 float 的类型转换，返回结果的 32 位表示。根据 IEEE754 单精度格式将 32 位划分为三段：**第 31 位为符号位 s** ，**第 30~23 位为阶码 E （8 位）**，**第 22~0 位为尾数 $frac$ （23 位）**，其表示的数值为 $(-1)^s \times 1.frac \times 2^{(E-127)}$ 。由于规格化数的有效数字最高位恒为 1，该位隐含不存储，因此实际精度为 24 位。而 int 的有效位数最多为 31 位，超出部分必须舍入。故本题的核心任务是：确定符号、计算阶码、提取尾数、处理舍入。

二、算法设计

- **第一步，特判 0。**浮点数 0.0 的表示为全 0，且后续规格化循环对 0 无法终止，因此需单独返回。
- **第二步，分离符号。**取 x 的最高位得到符号位，若为负则对数值取绝对值，后续只处理其绝对值大小。
- **第三步，规格化并计算阶码。**不断将绝对值左移，直至最高位（第 31 位）为 1，此时该数可视为 $1.xxx \times 2^{31}$ 。若无需左移，则真实阶数为 31，偏置后的阶码为 $31+127=158$ （之所以是 127 是因为会有大于 0 或小于 0 的指数，为平均表示（8 位只能表示 255 个数）所以用 ± 127 来表示；每左移一位说明原数的最高有效位低一位，阶码相应减 1。因此令阶码初值为 158，循环中每次左移后减 1，循环结束即得到正确阶码。
- **第四步，提取尾数。**规格化后，第 31 位是隐含的前导 1，第 30~8 位恰为 23 位尾数，第 7~0 位是超出精度的部分。将该数右移 8 位后与 **0x7FFFFFFF 相与（有 23 位 1，其他是 0）**，即可去掉隐含位、保留 23 位尾数。
- **第五步，舍入处理。**被舍去的低 8 位记为 rnd ，其“一半”为 0x80（因为 256 进一，所以一半是 128）。按 IEEE754 规定的“向偶数舍入”规则：当 rnd 大于 0x80 时进位；小于时直接截断；恰好等于 0x80 时，仅当尾数末位为 1（即结果为奇数）才进位，使结果取偶。
- **第六步，拼接结果。**三部分分别占据互不重叠的位段，将符号位、阶码左移 23 位后的值与尾数相加即可。**此处使用加法而非按位或，是为了正确处理舍入进位**：当尾数为全 1 时进位后产生第 23 位的溢出，加法会自动将其累加到阶码上，尾数归零，这恰好对应 $1.11...1$ 进位为 $10.00...0$ 、阶数加 1 的数学事实；若使用按位或则会丢失该进位。

10.floatScale2

一、预备知识：单精度浮点数的三种类型

- 规格化数对应阶码取 1 至 254，其数值为 $(-1)^s \times 1.frac \times 2^{(exp-127)}$ 。此类数的有效数字最高位恒为 1，该位隐含不予存储；阶码采用偏置

127 的方式编码，使得指数可正可负，实际范围为-126 至 127。绝大多数浮点数属于此类。

- **非规格化数**对应阶码为 0，其数值为 $(-1)^s \times 0.\text{frac} \times 2^{-126}$ 。此时隐含位为 0 而非 1，指数固定为-126。该类数用于表示绝对值小于最小规格化数的极小值，使浮点数在 0 附近能够平滑过渡，即所谓渐进下溢；当尾数亦为 0 时，表示 ± 0.0 。
- **特殊值**对应阶码为 255，其中尾数为 0 时表示 $\pm\infty$ ，尾数非 0 时表示 NaN（非数）。

二、分类依据与各类的处理方式

- 本题要求返回 2f 的位级表示。由于 $2^n \times 2 = 2^{n+1}$ ，乘 2 在数学上等价于指数加一；但三类数表达指数的方式不同，处理方式也随之不同。
- 判别的唯一依据是阶码字段，故首先将其取出：先右移 23 位使阶码对齐至最低位，再用 8 位掩码截取。据此分为四种情形。
- **其一，阶码为 255**。无论是无穷大还是 NaN，乘 2 的结果都与自身相同，题目也明确要求 NaN 原样返回，故直接返回原值。
- **其二，阶码为 0**，即非规格化数。此类数的指数固定为-126，无法通过修改阶码实现乘 2，只能将尾数本身翻倍，即整体左移一位。值得注意的是，若尾数最高位为 1，左移后将溢出至阶码字段，使阶码由 0 变为 1，恰好得到最小的规格化数。这一结果是正确的：IEEE754 在设计上保证了最大非规格化数与最小规格化数在数值上连续衔接，因此该溢出无需特殊处理，反而正是格式设计的自然体现。
- **其三，阶码为 254**，即规格化数中的最大阶码。加一后阶码变为 255，已超出规格化数的表示范围，按标准应返回同号的无穷大，即阶码全 1、尾数全零。此情形必须单独判断，否则将保留原尾数，得到阶码为 255 且尾数非 0 的位模式，那是 NaN 而非无穷大，结果错误。
- **其四，其余规格化数**，即阶码为 1 至 253。符号与尾数均不改变，只需将阶码加一。

三、方法一：利用位布局直接运算（搜索时接触到的）

```
unsigned floatScale2(unsigned uf) {
    unsigned s, exp;
    s = uf & 0x80000000;
    exp = (uf >> 23) & 0xFF;
    if (exp == 0xFF) return uf;
    if (exp == 0) return s | (uf << 1);
    if (exp == 0xFE) return s | 0x7F800000;
    return uf + 0x800000;
}
```

- 第一种实现不显式拆分各字段，而是利用字段在 32 位中的位置关系，对整体直接运算。
- 其关键在于对规格化数的处理：阶码字段的最低位在整个 32 位中对应权值 2^{23} ，因此对整个数加上常数 0x800000（即 1 左移 23 位），效果恰为阶码加一，而符号位与尾数位不受任何影响。相比“取出阶码、加一、再移位拼回”的三步操作，这里压缩为一次加法。
- 非规格化数的处理同样利用了位的连续性：将整个 32 位左移一位，尾数自然翻倍，若发生进位则直接落入阶码字段的最低位，自动完成向

规格化数的过渡，无需任何额外判断。由于左移会将符号位挤出，需用预先保存的符号位通过按位或恢复。该做法对 ± 0.0 同样成立，因为 0 左移仍为 0，符号得以保留。

- 该方法约使用 10 个运算符。其优点是简洁高效，体现了对浮点数位布局的整体把握；缺点是可读性较弱，加常数为何等价于阶码加一、进位为何恰好正确，都需要读者具备对位权值的理解。

四、方法二：字段拆解与重组（作业提交方法）

- 第二种实现遵循“拆解—判别—修改—重组”的通用流程：先用移位与掩码将 32 位分解为符号、阶码、尾数三个独立字段，再按数值类别分别修改，最后用移位与按位或三者拼回。
- 其中**取字段的通用式为“右移至起始位后与等宽的全 1 掩码相与”**，**重组的通用式为“各字段左移至其起始位后按位或”**。由于三个字段占据互不重叠的位段，按位或即可完成拼接而不会互相干扰。
- 与方法一相比，本实现将所有隐式的进位关系全部显式化：非规格化数的尾数左移后，显式判断是否产生了进位，若有则将阶码置为 1 并清除该溢出位；规格化数的阶码加一后，显式判断是否达到 255，若是则将尾数清零以构成无穷大。其最终结果与方法一完全一致，但每一步都与 IEEE754 的定义直接对应，便于逐条验证。
- 该方法约使用 18 个运算符，同样满足不超过 30 个的限制。

五、两种方法的比较

- 两种方法的差别不在算法，而在抽象层次。方法一将 32 位视为一个整体，借助字段位置的连续性，让加法与移位自动完成进位与字段迁移；方法二将 32 位视为三个独立字段的组合，把所有跨字段的影响显式写出。

11.float64_f2i

一、double 的格式

- [符号 1 位][阶码 11 位][尾数 52 位]
- 数值 $=(-1)^s \times 1.\text{frac} \times 2^{(\text{exp}-1023)}$ ，偏置是 1023（float 是 127）
- 52 位尾数拆在两个参数里：uf2 的低 20 位是尾数的高 20 位，uf1 的 32 位是尾数的低 32 位
- uf2:[s][exp11 位][frac 高 20 位]
- uf1:[frac 低 32 位]

二、问题分析

- 本题要求在不使用浮点运算的前提下，仅通过位运算完成 64 位双精度浮点数到 32 位有符号整数的转换，且舍入方式为向零截断。由于 C 语言中 unsigned 类型只有 32 位，无法直接容纳 64 位数据，题目将该浮点数拆为两个参数传入：uf2 为高 32 位，uf1 为低 32 位。
- 所谓“转换”，本质是两件事的同时发生：一是数值上丢弃小数部分，二是编码方式由浮点的“指数加尾数”形式改为整数的补码形式。因此不能简单截取位串，而必须先从浮点编码中还原出数值信息，再按整数格式重新构造。

三、总体思路

- 由于最终结果的符号可以在最后统一处理，整个计算过程只针对绝对值进行，这样也保证了截断方向始终朝向零。核心步骤可归纳为四步：**读出符号与指数、依据指数判断是否越界、将尾数还原为整数、按指数定位小数点。**
- **第一步，提取符号与阶码。**符号位是整个 64 位数的最高位，即 uf2 的第 31 位，右移 31 位即可取出。由于 uf2 为无符号类型，此处为逻辑右移，结果为 0 或 1，正好可直接作为判断条件。阶码位于 uf2 的第 30 至 20 位，先右移 20 位对齐至最低位，再与 11 位全 1 的掩码相与取出，减去偏置 1023 后得到真实指数 E。E 刻画了该数的量级，即其绝对值落在 2^E 与 $2^{(E+1)}$ 之间。
- **第二步，依据 E 判断边界情形。**若 E 小于 0，说明该数的绝对值小于 1，向零截断后结果必为 0，直接返回 0。**非规格化数与±0 的阶码为 0，减去偏置后 E 为-1023，同样落入此分支，无需单独处理。**若 E 大于 30，则该数的绝对值不小于 2^{31} ，超出 32 位有符号整数的表示范围，按题目约定返回 0x80000000。**NaN 与无穷的阶码为全 1，对应 E 为 1024，亦落入此分支，处理方式与溢出一致，符合要求。**此处返回 0x80000000 既是题目的规定，也与 x86 浮点转换指令在无效转换时的行为一致；此外，-2147483648.0 的 E 恰为 31 而被判为溢出，但 0x80000000 正是该值的补码表示，结果反而正确，故无需为其增设分支。
- **第三步，将尾数还原为一个 32 位整数。****补上被隐含省略的整数位 1**（二进制只有 0 和 1 两个数字，而科学计数法又规定开头不能是 0（否则就该继续挪小数点），所以开头只能是 1。既然每个数都一样，存它就是浪费。省下这 1 位，可以多存一位精度。这就是"隐含位"的由来。），令其落在第 31 位，随后依次填入尾数的高位部分：uf2 的低 20 位左移 11 位置于第 30 至 11 位，uf1 的高 11 位（即右移 21 位后的结果）置于第 10 至 0 位。三部分按位或即得到一个 32 位整数 m，其含义是有效数字 1.frac 放大 2^{31} 倍后的值。uf1 中更低 21 位代表更小的小数位，因最终要向零截断，直接舍弃不影响结果。这一步本质上仍是"字段拆解与重组"，只是字段跨越了两个参数。
- **第四步，按指数定位小数点。**m 表示的是 $1.frac \times 2^{31}$ ，而所求为 $1.frac \times 2^E$ ，二者相差 $2^{(31-E)}$ 倍，**故将 m 右移 31-E 位即可。**由于 E 的取值已被限定在 0 至 30 之间，移位量在 1 至 31 之间，不会出现未定义的越界移位。m 为无符号数，右移时高位补 0，不受符号扩展干扰；而被移出的低位恰为小数部分，被直接丢弃而不产生进位，这正好实现了题目要求的向零舍入。
- **最后，恢复符号。**若符号位为 1，对结果取相反数。之所以必须在取绝对值的基础上最后取负，是因为有符号数的算术右移采用向负无穷取整的方式：若先将负数参与移位，-13.75 将被截断为 -14，而题目要求的向零舍入应得 -13。先按绝对值截断再取负，可保证截断方向恒朝向零。

四、关键点小结

- 本题的难点集中在三处。其一是跨参数拼接尾数，需要清楚 52 位尾数在两个 32 位参数中的分布；其二是用指数 E 统一处理三类边界，使下

溢、溢出、NaN 与无穷共用两个判断即可覆盖；其三是舍入方向，必须通过"先取绝对值、后加符号"的顺序，配合无符号右移的截断特性来保证向零舍入。

2、floatPower2

一、问题的特殊性

- 本题要求返回 2.0^x 的单精度浮点位表示。与一般的浮点运算不同，2 的整数次幂在 IEEE754 中具有极其规整的形式：由于其有效数字恰好为 1，规格化表示下尾数字段全为 0，非规格化表示下尾数字段中仅有一位为 1。因此本题不涉及任何乘法或尾数计算，**只需判断 x 落在哪一区间，再把那个唯一的 1 放到正确的位置上即可。**

二、按 x 的取值划分四个区间

- 单精度浮点数能表示的数值范围有限， 2^x 随 x 增大或减小会分别越出上界与下界，中间又跨越规格化与非规格化两个区域，故需分四段讨论。分界点完全由格式的字段宽度决定。
- **其一，规格化区间：** $-126 \leq x \leq 127$ 。规格化数的值为 $1.\text{frac} \times 2^{(\text{exp}-127)}$ 。要使其等于 2^x ，只需令尾数全为 0（此时 1.frac 恰为 1），并令 $\text{exp}-127=x$ ，即阶码 $\text{exp}=x+127$ 。由于阶码字段仅能取 1 至 254（0 与 255 已被非规格化数和特殊值占用），对应的 x 范围即为 -126 至 127 。实现上只需将 $x+127$ 左移 23 位置入阶码字段。
- **其二，上溢区间：** $x > 127$ 。此时超出规格化数所能表示的最大指数，按题目要求返回 $+\infty$ ，其位模式为阶码全 1、尾数全 0。
- **其三，非规格化区间：** $-149 \leq x \leq -127$ 。非规格化数的值为 $0.\text{frac} \times 2^{-126}$ ，**其指数固定为 -126 ，无法再减小，只能通过尾数中 1 的位置来表示更小的数。**尾数字段共 23 位，自最高位（第 22 位）起依次代表 2^{-1} 、 2^{-2} 、……、 2^{-23} ，即第 k 位代表 $2^{(k-23)}$ 。将其与固定因子 2^{-126} 相乘，得到第 k 位所表示的数值为 $2^{(k-23-126)}=2^{(k-149)}$ 。令其等于 2^x ，解得 $k=x+149$ 。由于 k 的合法取值为 0 至 22，对应的 x 范围恰为 -149 至 -127 。实现上只需将 1 左移 $x+149$ 位。
- **其四，下溢区间：** $x < -149$ 。此时比最小的非规格化数 2^{-149} 还小，无法表示，按题目要求返回 0。

三、关键常数的来源

- 本题涉及两个"魔数"，均可从格式定义推出，不需死记。
- 127 是单精度阶码的偏置值，来源于 8 位阶码需要同时表示正负指数，故整体平移 127。
- 149 则由两部分构成：23 来自尾数字段的宽度，用于把"字段中第 k 位"折算为"小数点后第 $23-k$ 位"所代表的权值；126 来自非规格化数固定的指数 -126 。二者相加即 149。

四、实现要点

- 四个区间用连续的条件判断依次排除即可，每个判断只需写出一侧的边界，另一侧由前面的判断自动保证，从而简化代码。判断顺序必须由大到小：若不先排除 $x > 127$ ，阶码相加后会溢出至符号位；若不先保证 $x \geq -149$ ，左移位数为负数，属于未定义行为。